

Vivado Software Tutorial

This Document elaborates on the Vivado design suite software. Compatible with Vivado 2016.4 version.

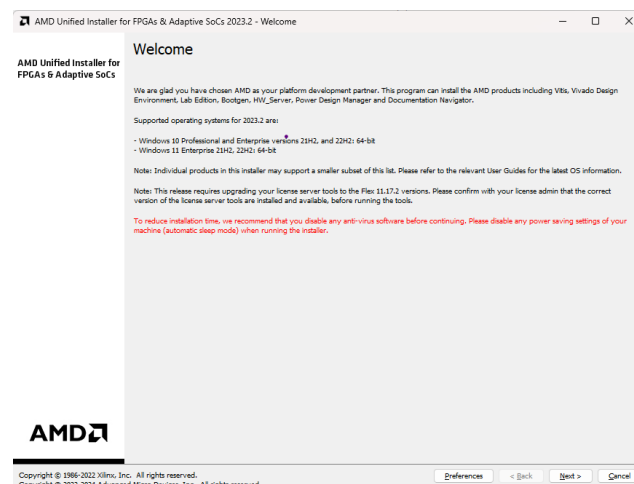
Contents

1. Vivado Installation	1
2. Vivado Project Creation	4
3. Constraints file (XDC).....	6
4. Behavioral Simulation.....	7
5. Implementing the design and checking timing.....	8
6. Integrated Logic Analyzer Setup.....	9
7. Integrated Logic Analyzer Removal	12
8. Integrated Logic Analyzer Usage	12
9. Programming the FPGA	15
10. Warnings and Errors.....	18

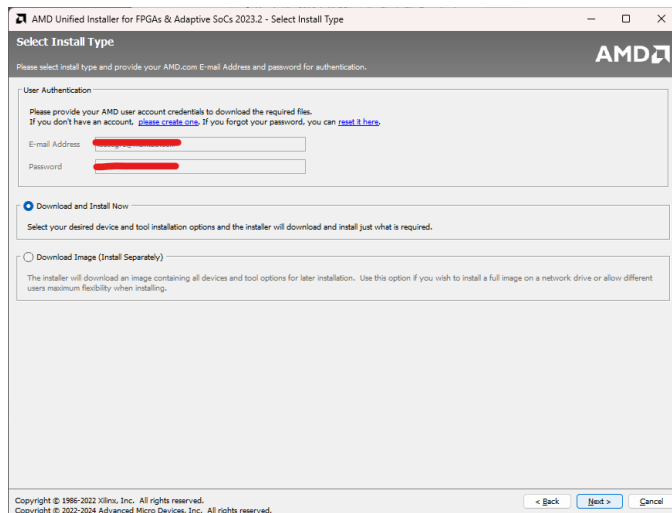
1. Vivado Installation

First step is to [download](#) the Vivado 2023.2 ML Edition software. After following the link above you will be directed to creation of a free Xilinx account. After creating an account and logging in, select the following version of Vivado.

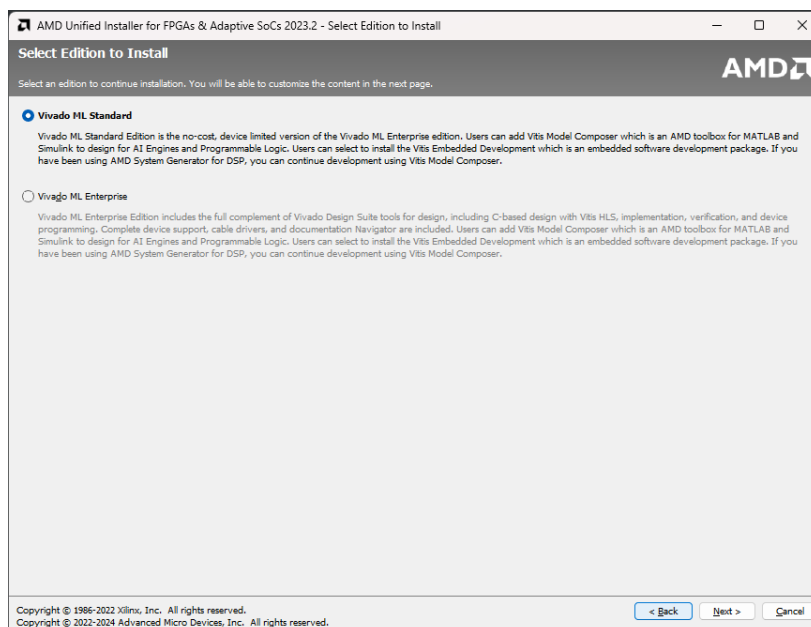
After the installer was downloaded, run it. You will be notified that a newer version of Vivado exists, ignore it by clicking “Continue” and then clicking “Next” in the wizard window:



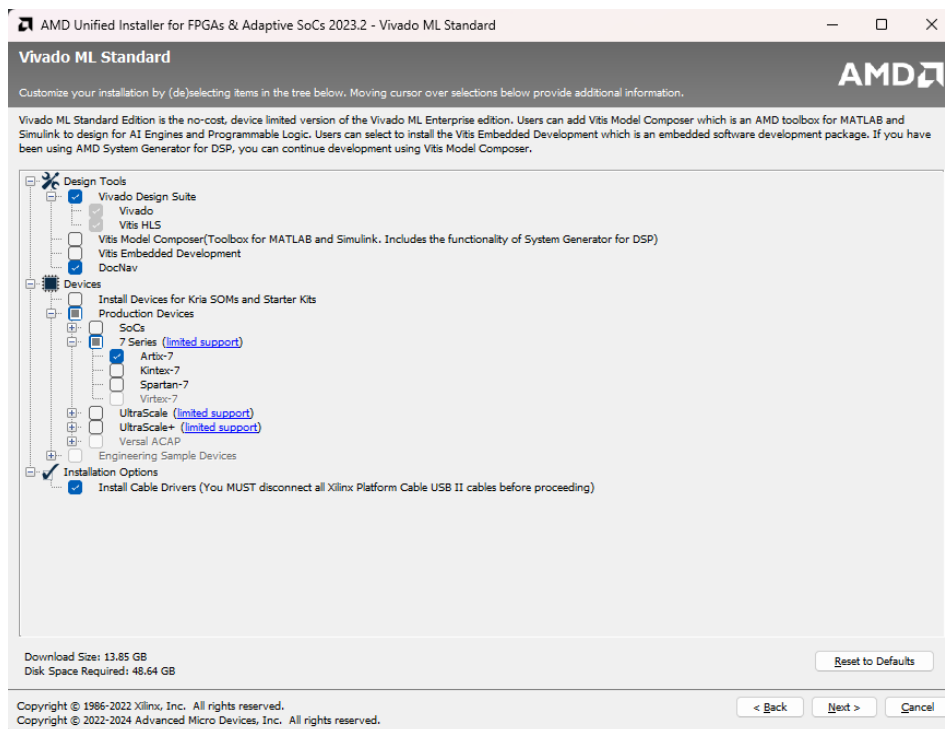
Enter your Xilinx account username and password. Make sure that “Download and Install now” radio button is chosen, and click “Next”:



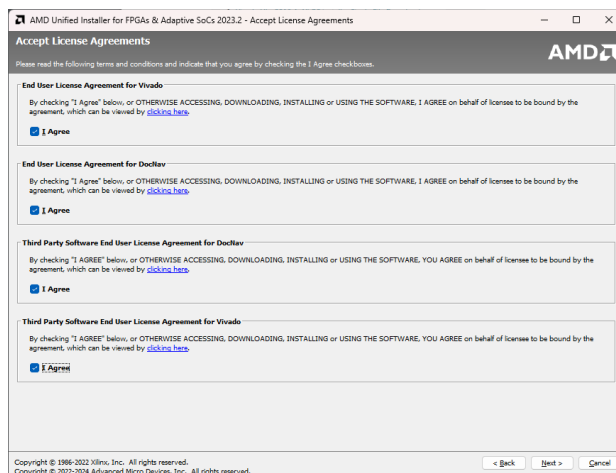
Next, choose the Vivado ML Standard and hit “Next”:



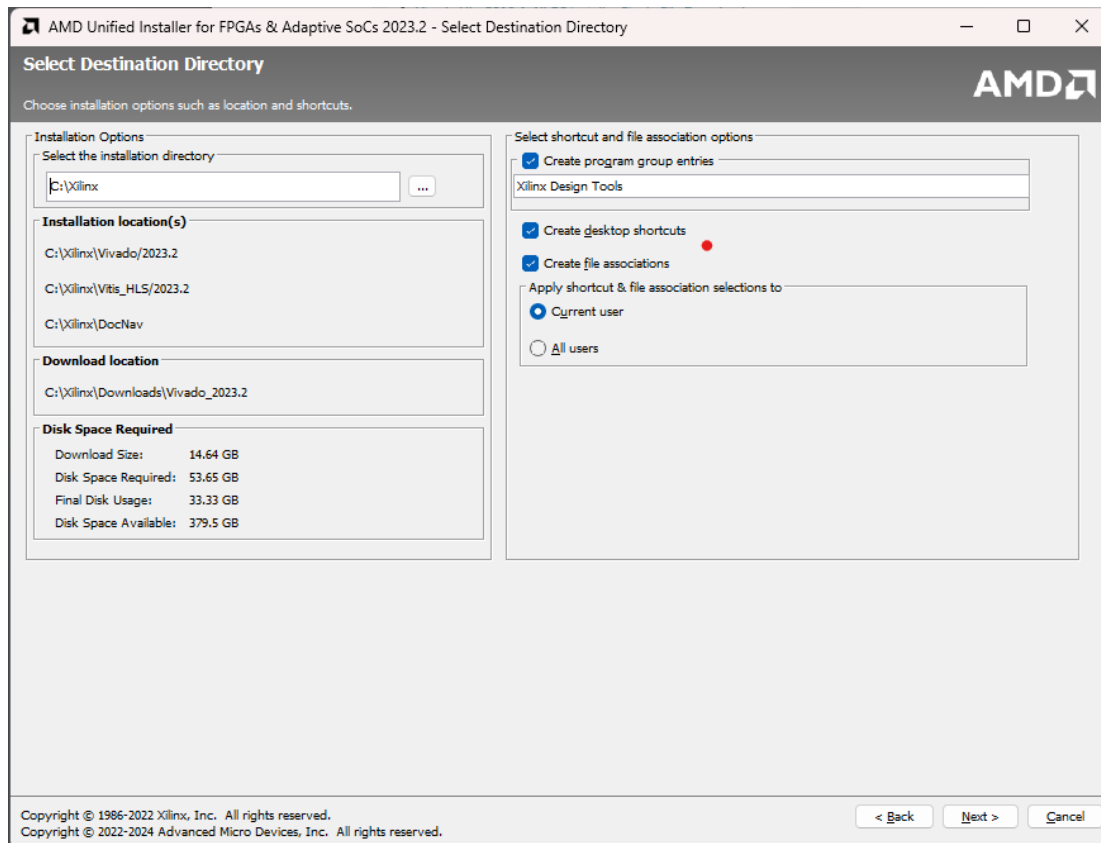
Next screen is important if you don't want to download redundant stuff. Take a careful look at the checkboxes checked in the next screen-shot. Make sure to have exactly the same checkboxes in your installation (most important for you is to keep the Artix-7 device checked), then hit “Next”:



In the following window, you have no choice but to accept all the terms. Resistance will be useless:



Make sure that you are cool with the installation path, click next:



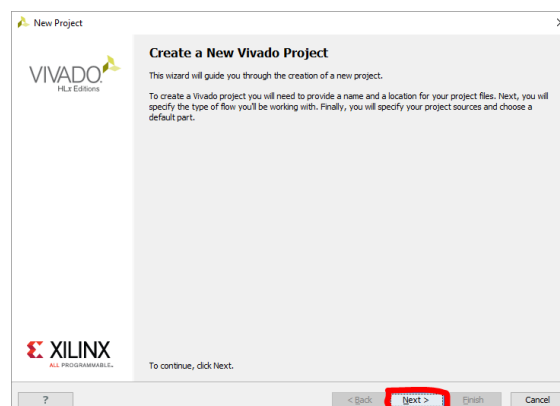
And in a screen after it click next again and you should be up and running.



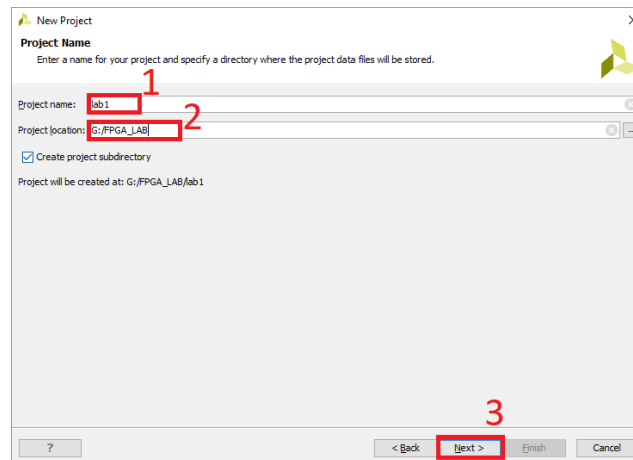
Popular installation bug: on some PCs, after installing Vivado, it won't run but continue showing messages with request to update Visual-Studio. Ignore these warnings, then go to the Vivado installation directory and remove the file named "xvcredist.exe".

2. Vivado Project Creation

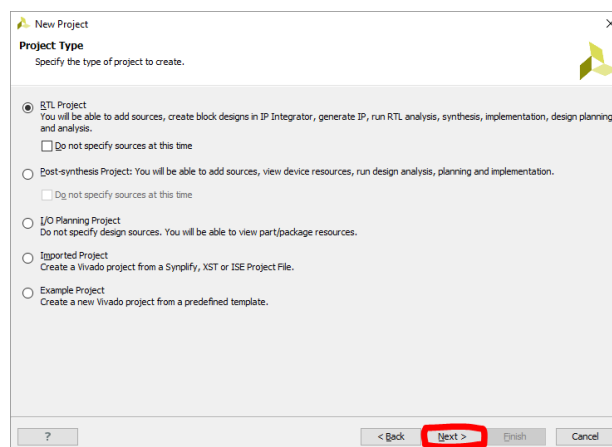
Launch the Vivado software, go to File→New Project. This will initiate a wizard that will guide you through the new project setup. First click on *Next*:



In the 2nd screen (Project Name) enter a project name, such as “lab1”, and specify a project location and click *Next*. **The project location must be on drive G if your work in the laboratory right now, and the path to it must not contain spaces or hebrew characters:**



In the 3rd screen (Project Type) leave the type as *RTL project* and make sure that the checkbox “Do not specify sources at this time” is unchecked. Click *Next*:



On the following 3 screens, you are given an option to add existing sources:

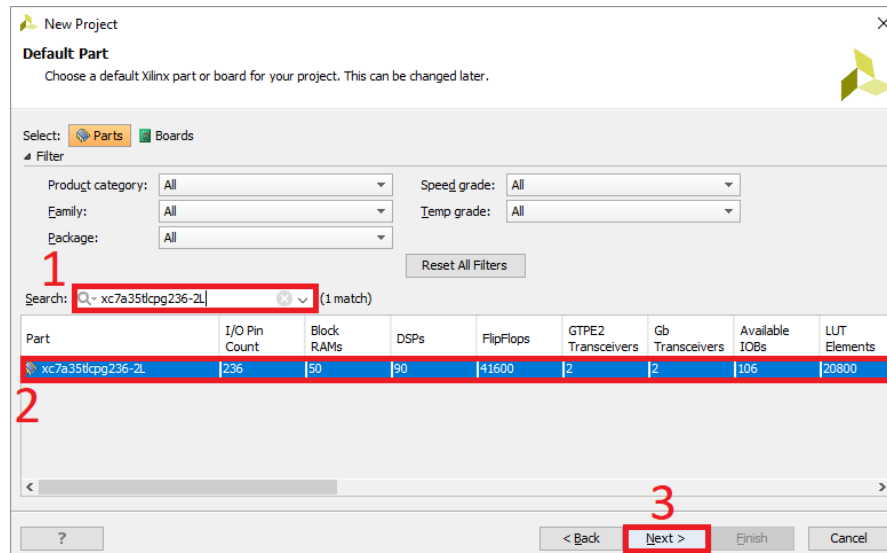
(1) Add design/simulation sources	(2) Add IP	(3) Add constraints
If you have some sources ready, you can add them now. Make sure to check the "copy sources..."	Will be mostly skipped in our lab.	Here you may add the .XDC file you received from the lab team. Make sure to check the "copy sources..."

checkbox, otherwise the sources will be left outside the project directory.

checkbox, otherwise the sources will be left outside the project directory.

To add more sources later, in the flow-navigator go to “Project Manager→Add Sources”.

In the “Default Part” screen, search for the catalog name **xc7a35tlcpg236-2L**. This is our chip:



Then you click Next, click Finish in the last “summary” screen and you are good to go.

3. Constraints file (XDC)

The design flow is almost fully automatic (automatic synthesis, implementations etc.), but there is a way for a designer to present his “special requests” from the automatic process. These special requests can be described in the constraints file (.xdc).

Xilinx provides numerous constraint types that can customize the synthesis, implementation, clocking and other automatized procedures of the design. The constraints file is also a place for several FPGA configuration mode settings such as the configuration voltage and rate.

For simplicity, in our lab we focus only on 2 types of constraints:

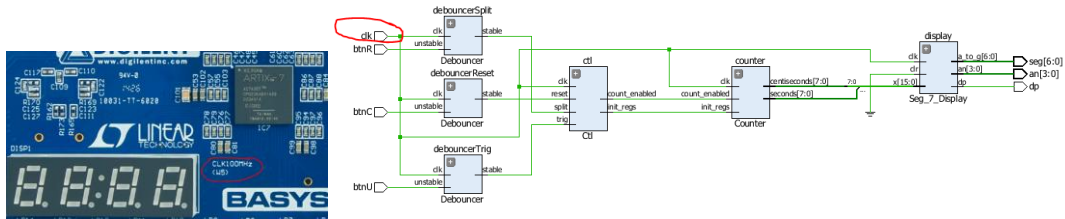
1. **Timing constraints** are criteria imposed on the timing analysis of your design. For example, the following line will define a clock constraint with a clock cycle time of 10 nanoseconds with a duty cycle of 50%

```
create_clock -period 10 -name sys_clk_pin -waveform {0 5} -add [get_ports clk]
```

After performing the timing analysis, you will see whether the timing constraints are met by going to (Synthesis/Implementation)→Report Timing Analysis in the Flow Navigator .

2. **IO constraints** – for interfacing between the IO ports of the Verilog top module and the IO of the FPGA. For example, the following two lines connect the "W5" pin of the chip (the output of the oscillator crystal) to the "clk" port of your design.

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

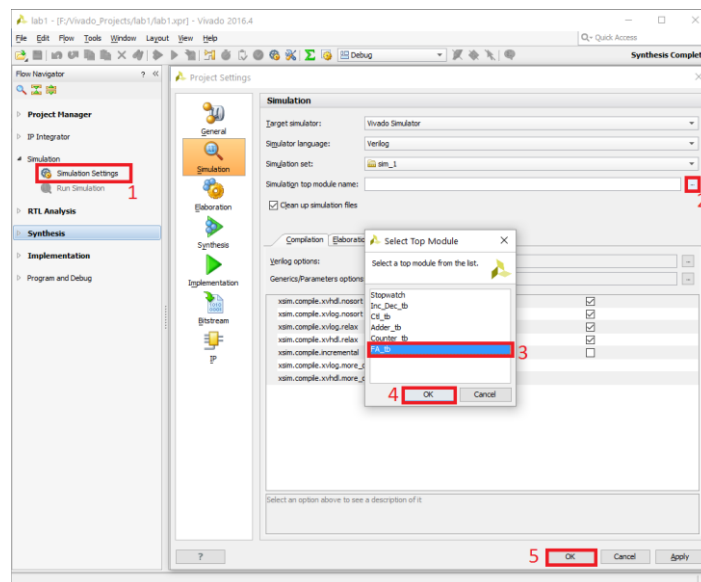


Normally, the ports that are not used in your design must be commented out.

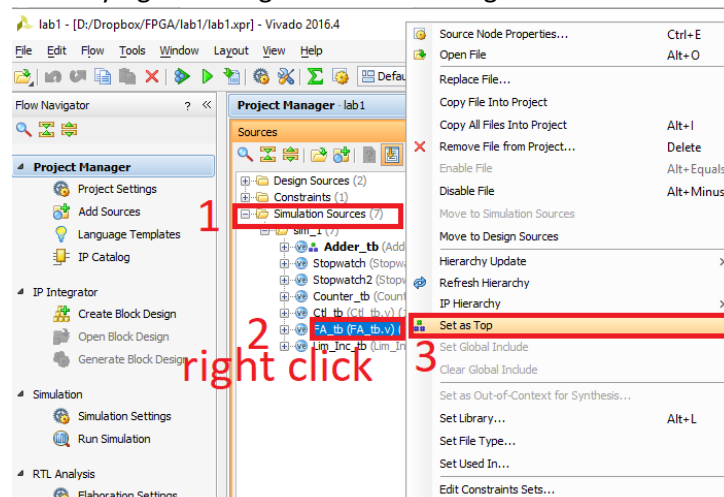
4. Behavioral Simulation

To test the logical correctness of your design, prior to undergoing the long process of synthesis and implementation, a good practice would be to perform a logical (aka behavioral) simulation. Suppose you have written a module FA.v and a test-bench module (which contains an "initial" block) called FA_tb.v.

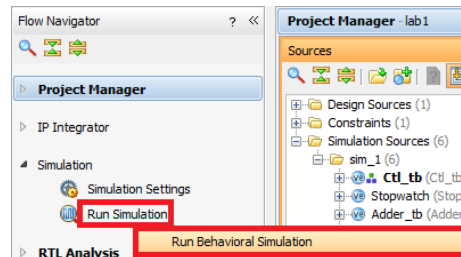
1. Choose the test bench that you wish to run, go to simulation settings:



Or alternatively: FA_tb.v (under the "Simulation Sources" folder) as the top test bench by right-clicking on it and choosing **Set as Top**:

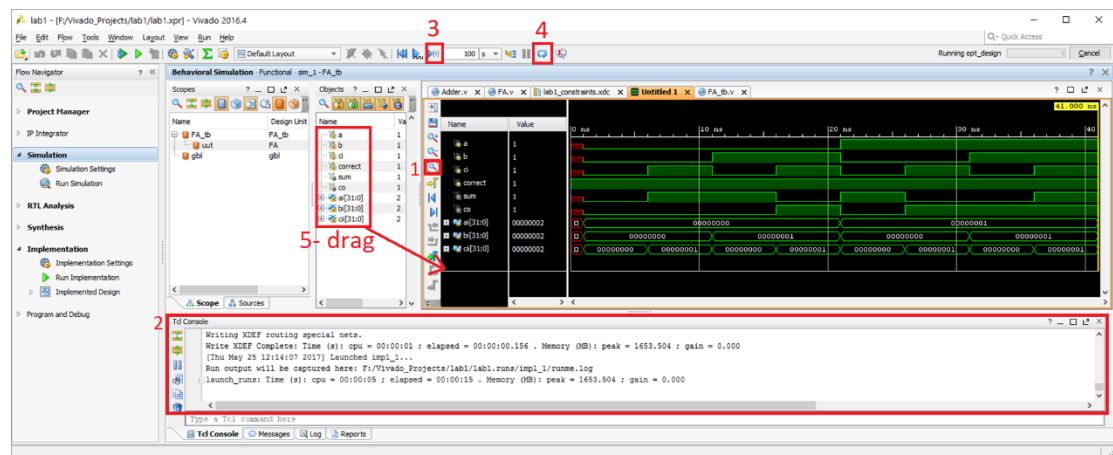


- Run the simulation. In the flow navigator go to Simulation→ Run Simulation→Run Behavioral Simulation



Popular problem when performing simulation: on some PCs, when clicking on “Run Behavioral Simulation”, Vivado will freeze with a message "Executing simulate step". This is commonly caused by the antivirus. So first try to disable antivirus to see if it's a problem, and if it is indeed the antivirus then find a way to add Vivado as an exception to Antivirus list.

- After a quick elaboration, a wave diagram view should open:

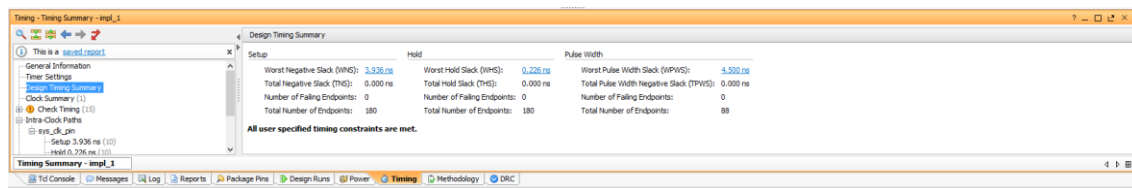


- Click on "zoom-fit", to set a decent time scale Otherwise, you will not see the whole waveform.
- Observe the TCL-console for errors.
- Run more time-steps of simulation, if needed (you might need to run more steps if \$finish command in your test-bench wasn't reached).
- Re-run the simulation from start – this is useful after you modify the test-bench or the UUT module, and you wish to re-run the simulation.
- You can pick more signals you want to inspect - by dragging them onto the waveform area.

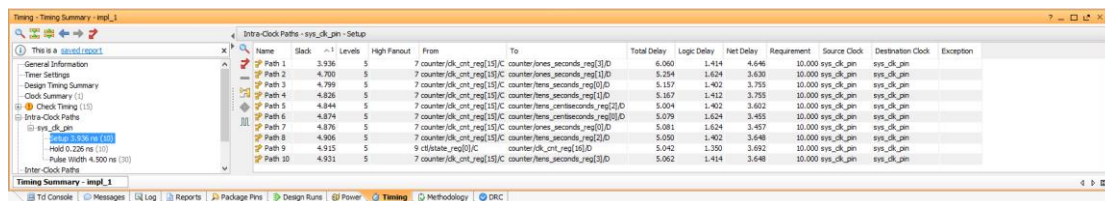
5. Implementing the design and checking timing

One can check timing analysis after synthesis, and in this case the path delays are computed over a synthesized netlist. If one decides to perform the timing analysis after the implementation – this will consider the final implemented netlist, including the FPGA's special placement and routing delays, which is much closer to the reality.

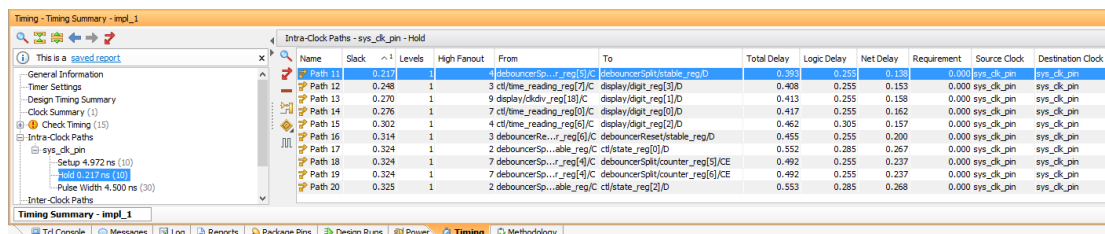
After performing Implementation→Run Implementation, always check for errors and critical warnings. After those were resolved, check the timing constraints in the lower screen, under "Timing" tab as follows:



This screenshot above shows that all the timing constraints were successfully met. If one would like to see the delays of specific paths in the logic, they can be shown as follows:



This screenshot above shows the "Setup paths". Which means the paths that could potentially violate the "setup" condition. For example, "Path 1" had a net-delay of 4.646ns and logic delay of 1.414ns, which summed up to 6.06ns total-delay. The analysis also shows us that there is a positive slack of 3.936, which means that we can hypothetically try to reduce the clock cycle by this amount of time and still retain a correct functionality.

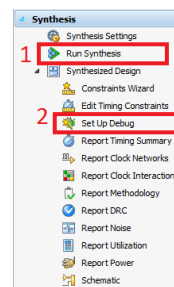


You can see that the hold paths can be violated in the opposite cases, of too short paths. Namely, the hold violation may happen when there is not enough delay on the path to keep the contaminating signal alive during the hold period.

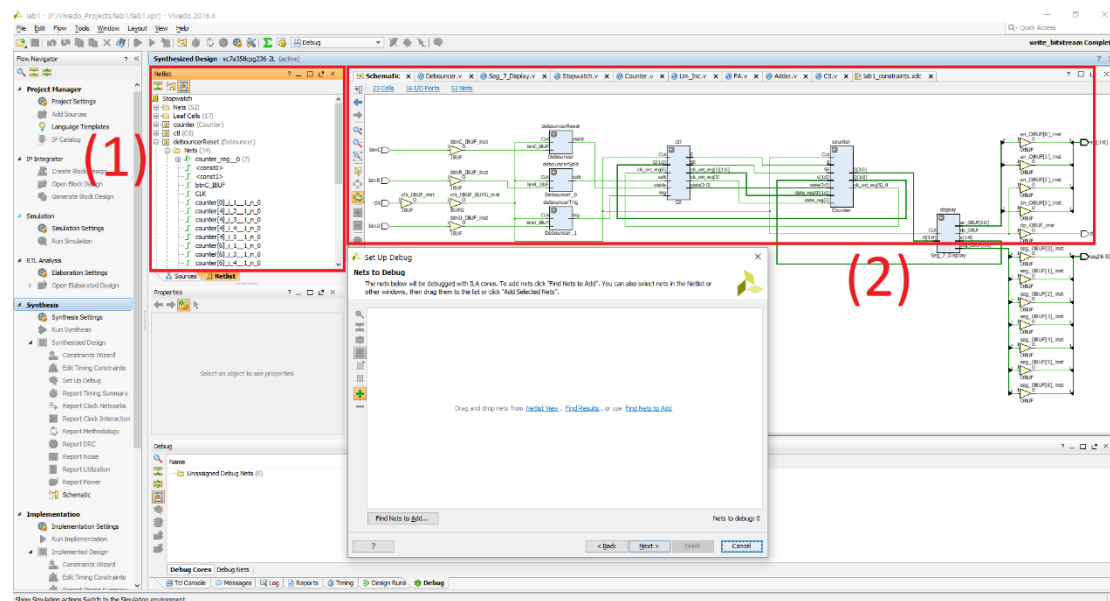
6. Integrated Logic Analyzer Setup

In order to allow a hardware debugging, the Vivado tool provides an integrated logic analyzer (ILA) which is added as a "debug core" to your design. The ILA is implemented alongside with your design and allows displaying your favorite internal signals (FF outputs, wires, etc) from the actual FPGA chip to the screen of the host PC.

To add an ILA to your design, after running the synthesis, in the flow navigator go to Synthesis→Set up debug



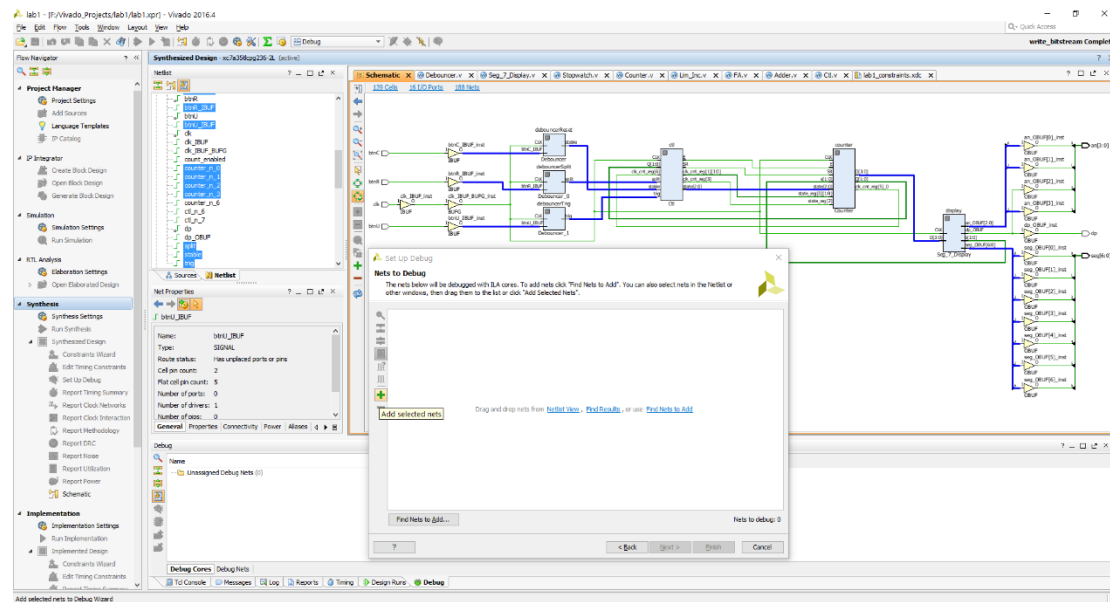
A wizard will pop up, click next. At the "Nets to debug" screen of the wizard – choose which nets would you like to be sampled by the ILA. It is equivalent to placing "probes" on them. You may either (1) pick up the nets from the netlist or (2) select them from the synthesized schematic view by holding Ctrl key + mouse clicking the nets:



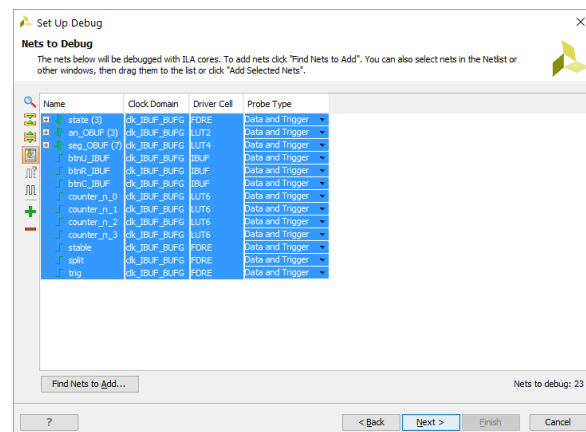
Notes:

1. Since you perform the addition of the ILA after the synthesis, you might not recognize the net names as you defined them, due to the optimization that often minimizes logic.
2. Don't choose clock signal, it will be recognized automatically
3. Don't choose the nets before the input-buffers and after the output-buffers, since these nets are non-debuggable.

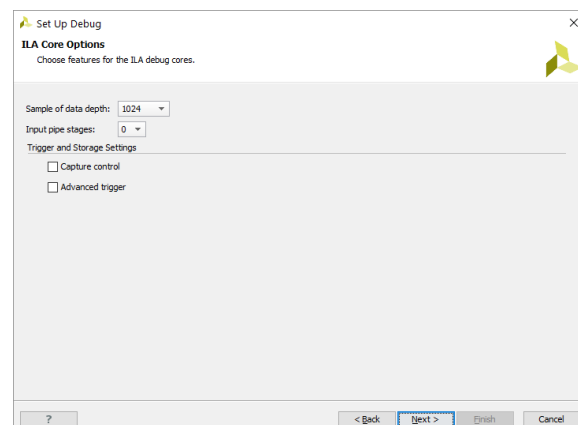
Suppose you selected the nets from the schematic (option 2 in the previous figure).



Then you click on  to add the selected nets:



Click next, and you reach the "ILA core options screen" where you choose the number of samples to be stored when trigger occurs. In the following figure, we choose 1024 samples:

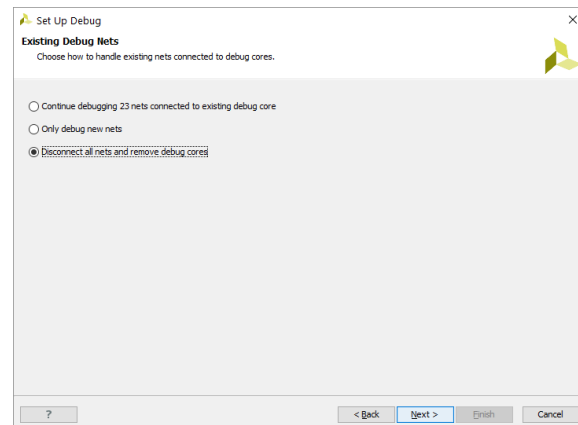


This means that enough on-chip memory will be allocated for storing all these 1024 samples of all the signals you probe. The more complicated your logic / more signals you sample, the more likely you will run out of resources and eventually the “design placement” will fail.

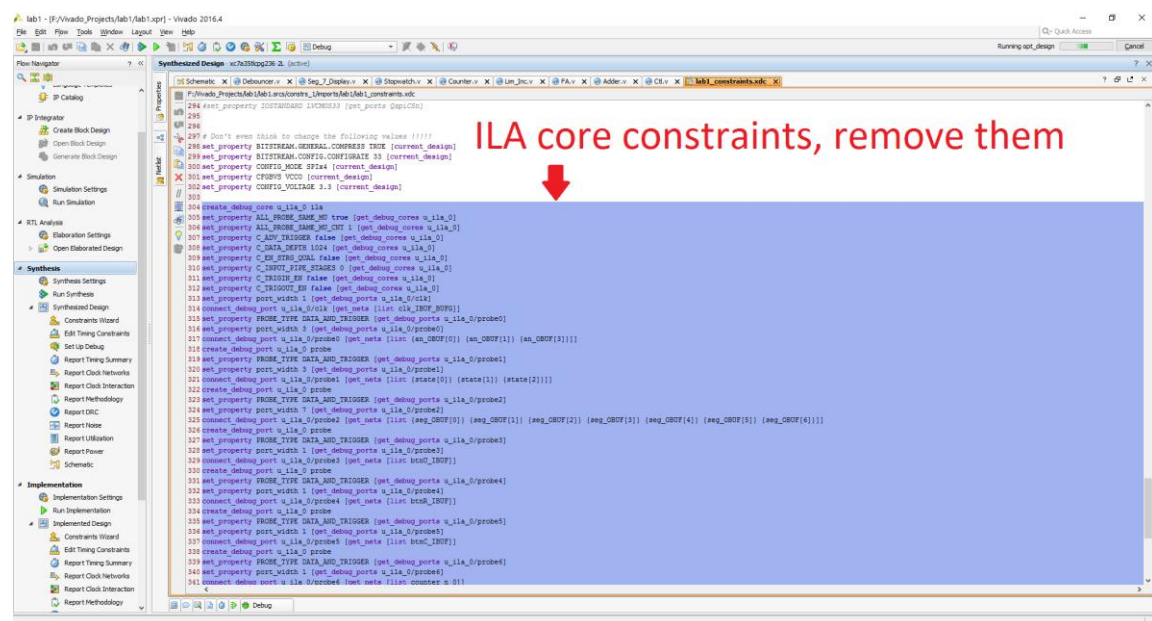
Then you click next and finish and you are supposed to see the debugged nets appear as dashed lines.

7. Integrated Logic Analyzer Removal

To remove the debug core of ILA, go to Synthesis→Set up debug and then in the first screen of the wizard choose the option to disconnect all nets and remove debug cores.

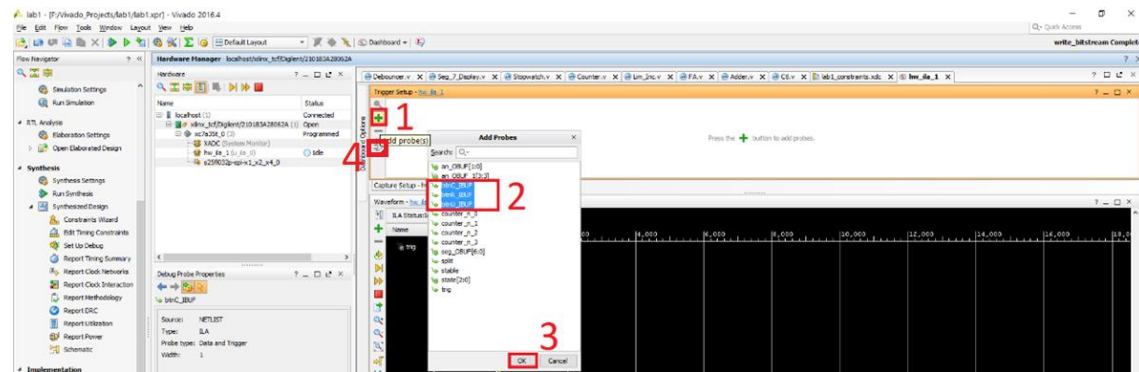


Then run synthesis again. If synthesis fails, you might need to manually remove the ILA realted lines from the xdc file (the line of create_debug_core u_ila_0 ila and everything below it):

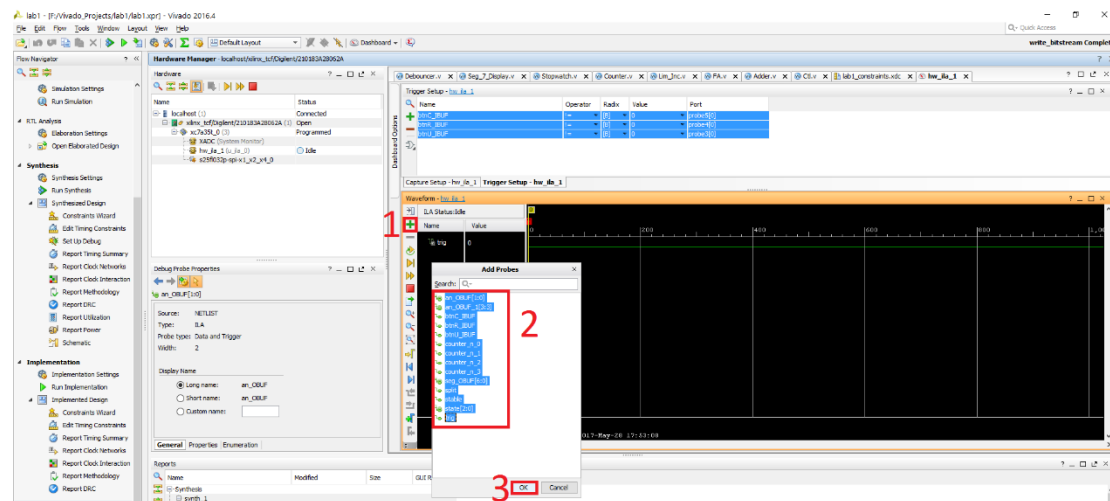


8. Integrated Logic Analyzer Usage

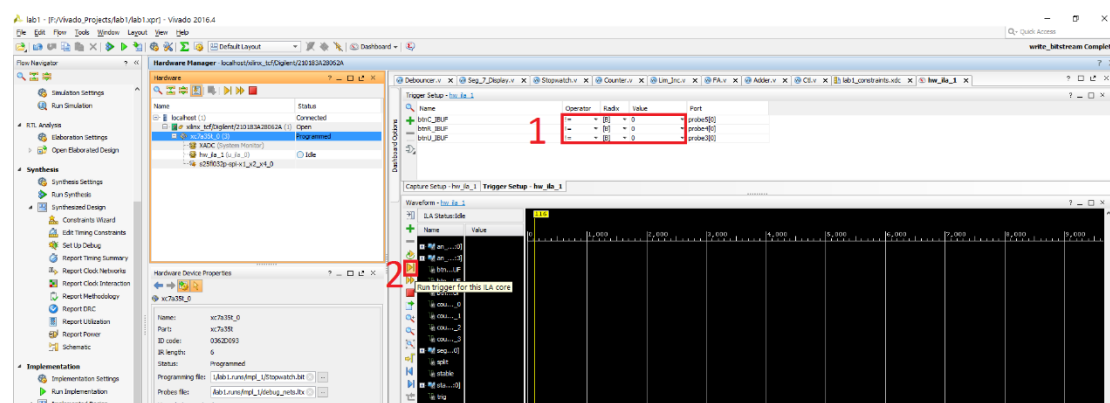
After a successful ILA setup, implementation and device programming, it is time to debug our design in hardware. The ILA core on your FPGA should be recognized automatically by the Hardware Manager and the following view should be opened upon a connection to the board, where you should (1,2,3) set up the trigger signals and then (4) set the trigger to 'OR' of the trigger signals, such that one of the triggers suffices a triggering:



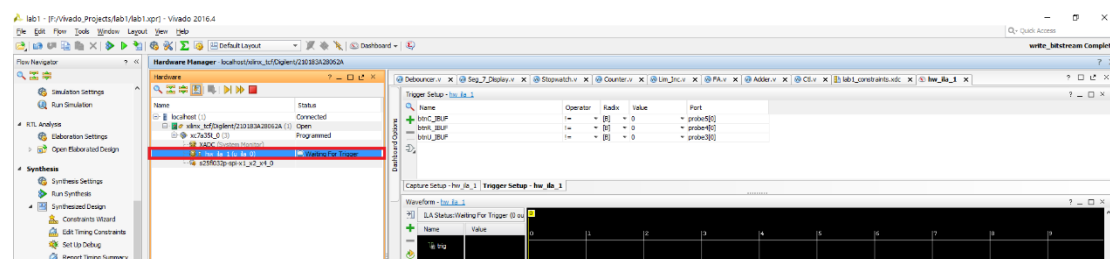
Add the signals desired for monitoring:



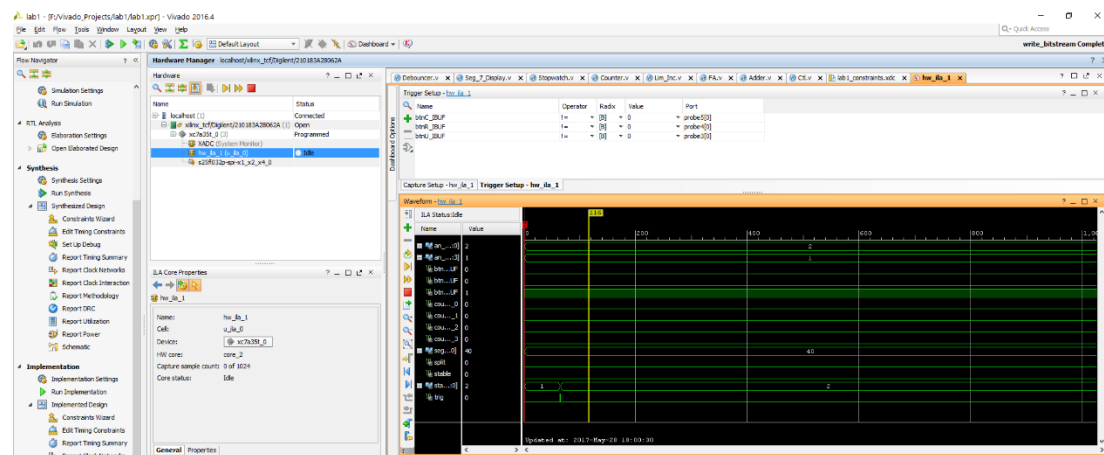
Then (1) set the triggers to react upon exiting from 0 level and (2) arm the ILA core



Your ILA core is now waiting for a trigger to come:



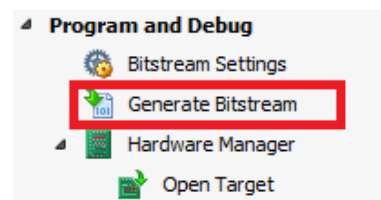
Hit one of the buttons that will activate the trigger and the samples from your debugged nets will appear on the wave diagram view:



9. Programming the FPGA

Step 1

After your design and constraints are synthesized and implemented without errors you may generate a bit-stream (.bit file):



The bit stream file of Artix-7 FPGA is typically 17,536,096 bits long, and can take time to be transmitted to the board. It can be compressed.

Step 2

Connect the BASYS3 board to the host PC USB port via the JTAG port on board.

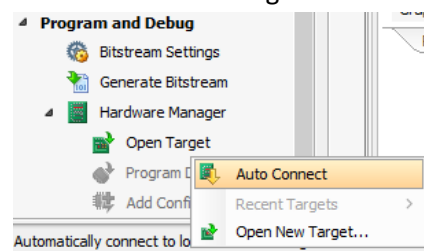
- Use the USB extender-cable we placed for you on the desk (it supposed to pop out from the monitor-cables hole in the desk)
- **When inserting the JTAG cable into its port on board. Pay attention to a correct orientation of the cable, as shown in the image below:**



Power on the board by switching the power switch:



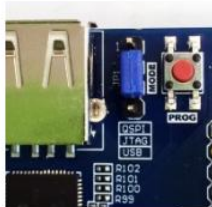
In the Vivado software, Program and Debug → Hardware Manager → Open Target → Auto Connect  to the target:



Step 3

The FPGA can be programmed in one of the 3 available ways, controlled by the position of the JP1 jumper. Extensive explanations can be found [here](#).

JTAG Programming



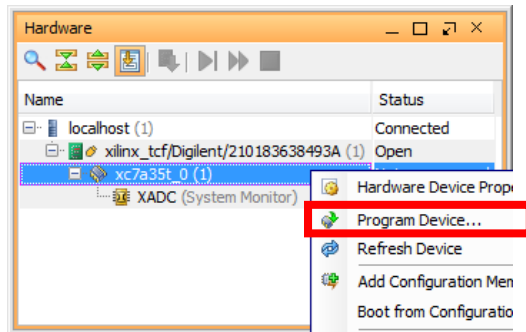
Quad SPI Programming



USB Host Programming



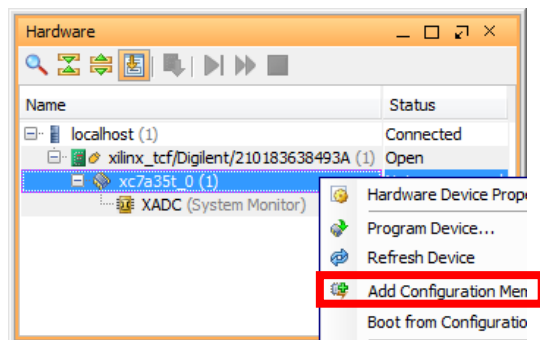
Jtag – [This is our preferred method] from PC (USB) to the board (J4 Jtag port). This programming is volatile (FPGA is unconfigured after the power off. To program via Jtag your bitstream file (.bit) must be generated, and then perform the following in the Hardware



Manager view:

Then choose the bit file (the path to it already supposed to appear in the appropriate field, if not, you can manually locate the file under lab1.runs/impl_1).

QSPI – [not used in our lab] This is a two-step programming. First, set the JP1 jumper to JTAG (upper position), then go to the project properties->Bitstream and check the *-bin_file* checkbox, after this you would need to re-generate the bit stream. After this:



Select the memory device:



Then agree to program the memory device (press OK), select the .bin file and press OK again.

The FPGA will be programmed and then the bin file is stored on the non-volatile. When the led 'done' is lit, power off the board, switch the JP1 jumper into QSPI mode (middle position) and power on the board - the FPGA is automatically programmed from the flash in less than a second.

USB – [not used in our lab] You can program the FPGA from a pen drive attached to the USB-HID port (J2) by doing the following:

1. Format the storage device (Disk-on-key drive) with a FAT32 file system.
2. Place a single .bit configuration file in the root directory of the storage device.
3. Attach the storage device to the Basys3.
4. Set the JP1 Programming Mode jumper on the Basys3 to "USB" (bottom position).

Push the PROG button or power-cycle the Basys3.

10. Warnings and Errors

This section elaborates on common **warnings**, **critical warnings** and **errors** you may encounter during synthesis, implementation and bitstream generation in Vivado.

- 1) [Synth 8-5788] Register X in module M is has both Set and reset with same priority. This may cause simulation mismatches. Consider rewriting code. This can be caused when your always@ block doesn't have a distinct separation for what happens in reset and what happens when there is no reset.
- 2) [Synth 8-689] width (N) of port connection 'X' does not match port width (K) of module 'M'. When you didn't define an appropriate number of bits for wire 'X' to fit the port of the module 'M' to which it was connected. Check your buses and registers!
- 3) [Synth 8-3848] Net 'X' in module/entity 'M' does not have driver. You connected a floating wire, or didn't connect anything at all to the port 'X' of module 'M'.
- 4) [Synth 8-3295] tying undriven pin M:X to constant 0. The synthesizer didn't find where you describe the functionality of port X in module M, it will be connected to 0
- 5) [Synth 8-3331] design M has unconnected port X. When you forget to attach wires to IO port X of module M.
- 6) [Synth 8-3332] Sequential element (M/X_reg) is unused and will be removed from module M. When register X of module M is actually not used, but you still insisted on declaring it. The synthesizer would remove this redundant flip-flop from the netlist.
- 7) [Synth 8-327] inferring latch for variable X. When a latch was inferred in a combinational logic . This is caused by using "if, else if, else if..." without terminating it with an "else" clause; or alternatively when you write an "case" statement without covering all cases or without using a "default" statement in the end.
- 8) [Synth 8-5788] Register X in module M has both Set and reset with same priority. This may cause simulation mismatches. Consider rewriting code. When no initialization value was properly set up for register X. Make sure that your synchronous always@ code blocks have a high-level structure of:

```
if (reset) begin ALL THE FLIP_FLOPS INITIALIZATIONS end  
else begin ALL THE LOGIC end
```


and that you initialize all your registers this way. You may receive this warning when this X register is assigned outside the if-else structure mentioned above.
- 9) [Vivado 12-584] No ports matched 'clk'. When your constraints file refers to Verilog ports that do not exist in your top-level Verilog module. Check that you marked the correct module as a top module.
- 10) ... multi-driven net Q ... When you accidentally performed an assignment to the same register from two separate always@ blocks.
- 11) Found timing loop. When there is a cycle of combinational logic, which is not allowed in digital logic design. Vivado will also specify a random point on this circular path and using this hint you should try to determine where the loop passes and introduce at least one flip-flop along it (flip-flops effectively finish the combinational path. This type of critical warning can be difficult to correct, as those circular paths often pass through several modules in your design.